Local Installation

This guide gets Candex running on your machine: the **API** with a local PostgreSQL database, the **mobile app**, and the **admin dashboard**. If you have not yet, skim Working on Candex first for the big picture.

The local database is built **from scratch** by the API's Flyway migrations (which include seed data), so you do **not** need access to any server.

What you will have running

Service	URL
REST API	`http://localhost:8080` (Swagger at `/swagger-ui.html`)
Mobile app	`http://localhost:8100`
Admin dashboard	`http://localhost:8101`
PostgreSQL	`localhost:5432`

Prerequisites

- **Git**
- **Node.js 20+** and **npm 10+** — for the app and dashboard
- **JDK 21** — for the API (a Maven wrapper, `mvnw`, is included, so a separate Maven install is optional)
- **Docker** and Docker Compose — for the local Postgres + API stack
- A **Firebase project**'s config — for the app's authentication (see the app section)
- **Android Studio** — optional, only for native Android builds

Get the code

Clone the repositories into one folder:

```
git clone https://github.com/RusterX16/candex-api
git clone https://github.com/RusterX16/candex-app
git clone https://github.com/RusterX16/candex-dashboard
```

1 — API and database (`candex-api`)

Create the environment file from the template and fill in values:

```
cd candex-api
cp .env.example .env     # then edit .env
```

The recommended path is the helper script, which runs a throwaway containerised stack (Postgres 17 + API) from `compose.local.yaml`:

```
./scripts/candex.sh up        # start local Postgres + API
```

On startup the API runs its **Flyway migrations**, which create the schema and seed baseline data (brands, app versions, and more) — a working database with no server access required.

Other useful commands:

```
./scripts/candex.sh down      # stop the stack (data kept)
./scripts/candex.sh reset     # wipe the local database volume
./scripts/candex.sh psql      # open psql on the local database
./scripts/candex.sh logs      # follow the API logs
```

The API is then at `http://localhost:8080`, with Swagger UI at `/swagger-ui.html`.

> **Without Docker**
>
> If you prefer to run your own PostgreSQL, set `CANDEX_DB_URL`, `CANDEX_DB_USERNAME`, and `CANDEX_DB_PASSWORD`, then start the API with `./run-dev.sh` (or `./mvnw spring-boot:run -Dspring-boot.run.profiles=dev`).

2 — Mobile app (`candex-app`)

Install dependencies:

```
cd candex-app
npm install
```

The app generates its Angular environment from env files (`scripts/generate-env.js`). Create them from the template:

- `.env.dev` and `.env.prod` — one Firebase project each, plus `API_URL` (copy `.env.example`)
- `.env.local` — just `API_URL=http://localhost:8080`, which overrides the API URL for local runs

Firebase config. The app authenticates through Firebase, so `.env.dev` / `.env.prod` need the seven `FIREBASE_*` values, and the Android build needs `android/app/google-services.json`. Get these from your Firebase project (Project settings → your web app) or from the project owner. No Firebase values are included in this documentation.

Run against your local API:

```
npm run start:local     # http://localhost:8100 (dev Firebase + localhost API)
```

`start:local` pairs with the API's **dev** profile — keep both on `dev`. For a native Android build (needs Android Studio): `npm run android:dev`.

3 — Admin dashboard (`candex-dashboard`)

```
cd candex-dashboard
npm install
cp .env.example .env     # set API_URL_LOCAL and the admin credentials
npm start                # http://localhost:8101
```

Putting it together

For a consistent local setup, keep the app's **local** mode and the API's **dev** profile together. Typical run order: start the API (`candex.sh up`), then the dashboard (`npm start`) and the app (`npm run start:local`).

Troubleshooting

- **Port already in use** — stop the process holding `8080`, `8100`, `8101`, or `5432`, or change the port.
- **API cannot reach the database** — make sure Docker is running and `candex.sh up` reports a healthy `db`; check the `DEV_DB_*` values in `.env`.
- **App build fails with "Missing env vars"** — `generate-env.js` needs `API_URL` plus all seven `FIREBASE_*` values in the loaded `.env.*` file.
- **Empty database** — migrations run when the API starts; check `candex.sh logs` to confirm Flyway applied them.

Where to go next

- API internals → Back-end *(planned)*
- Environments and releases → Deployment *(planned)*